Loss Landscapes & Optimizers

Learning Rate Schedules • Saddle Points & Local Minima • Gradient Clipping

Learning Rate Schedules

What is the Learning Rate?

During gradient descent, neural-network weights are updated using the gradient of the loss. The learning rate determines how large a step the optimizer takes in the direction that reduces the loss.

$$W_{new} = W_{old} - \eta \frac{\partial L}{\partial W}$$

Here, W is the weight, L is the loss, the derivative is the gradient, and η is the learning rate.

Intuition: imagine walking down a mountain toward the lowest point. A very large learning rate takes huge steps and may jump over the valley. A very small learning rate takes tiny steps and makes training slow. An appropriate learning rate reaches the bottom efficiently.

Very Large Learning Rate

A very large learning rate can cause the optimizer to overshoot the minimum, oscillate around it, fail to converge, or make the loss explode. In extreme cases, training can produce NaN values.

Very Small Learning Rate

A very small learning rate produces stable but tiny updates. Training may become extremely slow and may require many epochs to converge.

Why Use a Learning Rate Schedule?

Instead of keeping the learning rate fixed, a learning rate schedule changes it during training. A common strategy is to use a relatively large learning rate when the model is far from a good solution and a smaller learning rate when it approaches a good solution.

Example: 0.01 → 0.005 → 0.001 → 0.0005.

Major Learning Rate Schedules

Step Decay

The learning rate is reduced at predefined intervals.

- Epochs 1–10: 0.01
- Epochs 11–20: 0.001
- Epochs 21–30: 0.0001

$$\eta_t = \eta_0 \gamma^{\lfloor t/k \rfloor}$$

η_0 is the initial learning rate, γ is the decay factor, k is the interval, and t is the current epoch.

Advantages: simple and easy to implement. Disadvantage: the schedule must be chosen manually and may reduce the learning rate too early or too late.

Exponential Decay

The learning rate decreases smoothly over time.

$$\eta_t = \eta_0 e^{-kt}$$

Another common form is $\eta_t = \eta_0 \gamma^t$, where $0 < \gamma < 1$. Exponential decay is smooth, but the decay rate must be chosen carefully.

Reduce Learning Rate Based on Validation Performance

Instead of reducing the learning rate after a fixed number of epochs, we can reduce it when validation performance stops improving. ReduceLROnPlateau is a common example.

Example validation loss: 0.40, 0.32, 0.27, 0.25, 0.25, 0.25, 0.25. If the loss does not improve for several epochs, the learning rate can be reduced.

```
callback = tf.keras.callbacks.ReduceLROnPlateau(
    monitor="val_loss",
    factor=0.1,
    patience=3,
    min_lr=1e-6
)
```

Cosine Annealing

Cosine annealing gradually decreases the learning rate according to a cosine-shaped curve.

$$\eta_t = \eta_{min} + \frac{1}{2}(\eta_{max} - \eta_{min})\left(1 + \cos\left(\frac{\pi t}{T}\right)\right)$$

This provides a smooth reduction from a relatively high learning rate toward a lower value and is widely used in deep-learning training.

Warm-Up

Warm-up starts training with a small learning rate and gradually increases it before another schedule takes over. Example: 0.0001 → 0.0003 → 0.0006 → 0.001.

Warm-up can reduce early-training instability, especially when large updates at initialization could destabilize the model.

Saddle Points and Local Minima

Optimization is not difficult only because of local minima. In high-dimensional neural networks, saddle points and flat regions can also make optimization slow or difficult.

Local Minimum

A local minimum is a point where the loss is lower than nearby points. At a differentiable local minimum, the gradient is zero or approximately zero.

A global minimum is the lowest possible loss over the entire parameter space, while a local minimum is lowest only within a neighborhood.

Saddle Point

A saddle point is a point that behaves like a minimum in one direction and a maximum in another direction. A classic example is:

$$f(x, y) = x^2 - y^2$$

At (0,0), the partial derivatives are:

$$\frac{\partial f}{\partial x} = 2x$$

$$\frac{\partial f}{\partial y} = -2y$$

$$\nabla f(0, 0) = (0, 0)$$

Along the x-direction:

$$f(x, 0) = x^2$$

which has a minimum at x=0. Along the y-direction:

$$f(0, y) = -y^2$$

which has a maximum at y=0. Therefore, the point is a saddle point rather than a minimum.

Why Saddle Points Are Difficult

Near a saddle point, the gradient can be close to zero, so weight updates become very small and optimization can appear to be stuck.

$$W_{new} = W_{old} - \eta \frac{\partial L}{\partial W}$$

Neural networks can have millions or billions of parameters, creating a high-dimensional loss surface with many complex regions. Stochastic mini-batch gradients, momentum, and adaptive optimizers such as Adam can help the optimizer move through difficult regions.

Momentum and Adam Intuition

$$v_t = \beta v_{t-1} + \nabla L$$

$$W_{t+1} = W_t - \eta v_t$$

Momentum accumulates information from previous gradients, helping the optimizer maintain movement through flat or difficult regions. Adam combines momentum-like first-moment information with adaptive scaling based on gradient statistics.

Gradient Clipping

What is an Exploding Gradient?

If gradients become extremely large during backpropagation, the resulting weight updates can become enormous. This is called the exploding-gradient problem.

For example, if η = 0.001 and the gradient is 10,000, the update magnitude is 10, which may be far too large.

$$\Delta W = -\eta g$$

Exploding gradients can cause unstable training, loss oscillation, loss explosion, numerical overflow, NaN values, and divergence.

What is Gradient Clipping?

Gradient clipping limits the size of gradients before they are used to update weights. The main approaches are clipping by value and clipping by norm.

Gradient Clipping by Value

Each gradient component is restricted to a specified range. If the clipping range is [-2, 2], values such as -10 and 8 become -2 and 2.

- Original gradient: [-10, -4, -1, 0.5, 3, 8]
- After clipping to [-2, 2]: [-2, -2, -1, 0.5, 2, 2]

Because each component is clipped independently, the overall direction of the gradient can change.

Gradient Clipping by Norm

Norm clipping limits the magnitude of the entire gradient vector. Consider g = [3, 4].

$$\|g\|_2 = \sqrt{3^2 + 4^2} = 5$$

If the maximum allowed norm is c = 2, the gradient is scaled:

$$g_{clipped} = g\frac{c}{\|g\|}$$

$$g_{clipped} = (1.2,\ 1.6)$$

The overall magnitude is reduced while the direction is preserved. This is why norm clipping is often a natural first choice for exploding gradients.

Gradient Clipping in TensorFlow

```
optimizer = tf.keras.optimizers.Adam(
    learning_rate=0.001,
    clipnorm=1.0
)
```

clipnorm=1.0 limits the overall gradient norm. clipvalue=1.0 clips each individual gradient component to the range [-1, 1].

Clipnorm vs Clipvalue

Feature	Clipnorm	Clipvalue
What it limits	Overall gradient magnitude	Individual gradient components
Direction	Generally preserved	Can be altered
Typical use	Exploding-gradient control	Simple component-wise restriction
TensorFlow	clipnorm=1.0	clipvalue=1.0

What Gradient Clipping Does Not Fix

Gradient clipping is mainly intended to address exploding gradients. It does not directly solve vanishing gradients, poor architecture, bad initialization, an inappropriate learning rate, overfitting, underfitting, or poor data normalization.

How the Three Concepts Are Connected

- If optimizer steps are too large or too small, adjust the learning rate or use a learning-rate schedule.
- If optimization enters a flat or saddle region, momentum, stochasticity, adaptive optimizers, and suitable initialization can help.
- If gradients become excessively large, gradient clipping limits their magnitude.

Gradient vs Learning Rate

The gradient tells the optimizer which direction to move and how steep the loss surface is. The learning rate controls how large the movement should be.

$$W_{new} = W_{old} - \eta \frac{\partial L}{\partial W}$$

Gradient clipping modifies the gradient. Learning-rate scheduling modifies η.

Easy Intuition to Remember

Concept	Mountain/Driving Analogy
Gradient	Tells you which way the road slopes.
Learning rate	Determines how aggressively you move.
Learning-rate schedule	Move faster initially, then slow down near the destination.
Saddle point	Terrain slopes upward in one direction and downward in another.
Gradient clipping	Prevents an excessively large step when the gradient suddenly becomes huge.

Summary

Concept	Main Problem	Main Idea
Learning Rate	Step too large/small	Controls update size
Learning Rate Schedule	Fixed LR may not work throughout training	Change LR during training
Step Decay	Need periodic reduction	Reduce LR at predefined intervals
Exponential Decay	Need smooth reduction	Gradually decay LR
ReduceLROnPlateau	Validation loss stops improving	Automatically reduce LR
Cosine Annealing	Need smooth schedule	Cosine-shaped decay

Warm-up	Initial training instability	Start small and increase LR
Local Minimum	Gradient can become zero at a low point	May slow/stop optimization
Saddle Point	Gradient can be near zero without being a minimum	Optimization may move slowly
Exploding Gradient	Gradient becomes extremely large	Can cause unstable updates
Gradient Clipping	Exploding gradients	Limit gradient magnitude
Clip by Value	Individual gradients too large	Restrict each component
Clip by Norm	Overall gradient too large	Scale whole gradient